

CSE406: Cyber Security Sessional

Design Report

Stealth Port Scanning with Active OS / Version Fingerprinting

Attack Tool #8 — Port Scanning with OS Information / Version Detection

Students	Supervisor
Md. Sadrul Islam (2105018) Tanvir Liaquat Uday (2105026)	Imtiaz Kabir

1 Overview of the Project Idea

This project implements a two-stage reconnaissance tool built entirely from raw sockets in Python: (1) a stealth, half-open TCP port scanner that discovers open, closed, and filtered ports on a target host while minimizing the footprint left on the victim and any monitoring device, and (2) an active OS/version fingerprinting engine that crafts a small, custom battery of TCP/ICMP/UDP probes exploiting ambiguities left open by RFC 793, RFC 1122, and RFC 3168, then matches the observed response vector against a self-built signature database to output a ranked list of candidate operating systems with a confidence percentage, in the style shown in the project's own demo run (Figure 5).

The assignment's overriding requirement is fingerprinting accuracy, so stealth is applied only where it does not compromise the completeness of the probe set: the port-discovery phase uses half-open SYN scanning, randomized port ordering, randomized inter-probe timing (jitter), and optional decoy source addresses to blend in with background traffic and avoid completed-connection log entries; the OS-detection phase itself is short (well under a second per target) and is not further throttled, since additional probes are what make the classification confident rather than what makes it noisy.

The design is informed by two public references that are used only to understand which protocol behaviors are useful discriminators — the actual probe set, packet construction, matching logic, and signature database in this project are all implemented from scratch, as required: Fyodor's description of the TCP/IP stack tests used by Nmap's OS-detection engine, and Netresec's overview of passive TTL/window-size fingerprinting. See the References section.

As a validation strategy, the finished tool's OS-guess output will itself be benchmarked against Nmap's own OS-detection engine (`nmap -O`) run against the same lab targets: Nmap's result is treated as an independent reference point (not as ground truth, since Nmap can also be wrong), and the two tools' top candidates and confidence trends are compared to gauge how well a small, self-built probe set and signature database perform relative to a mature, heavily-calibrated fingerprinting database. Details are in Section 5.6.

2 Network Topology and Components

2.1 Components

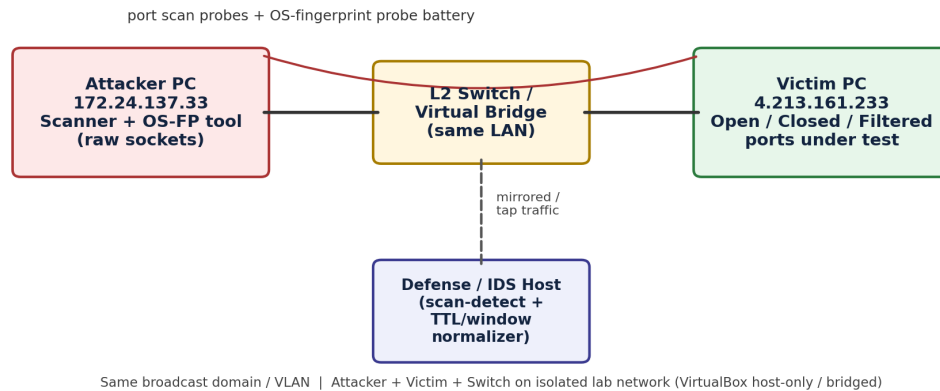
Figure 1 — Network Topology: Stealth Port Scan + OS Fingerprinting Testbed

Figure 1: Attacker, victim, and switch on an isolated lab LAN; an optional defense/IDS host taps mirrored traffic for Section 7.

Component	Role
Attacker PC	Runs the custom scanner and OS-fingerprint engine (Python, raw <code>AF_INET/SOCK_RAW</code> sockets with <code>IP_HDRINCL</code> , plus an <code>AF_PACKET</code> sniffer thread to correlate responses to sent probes).
Victim PC	Target host with a known ground-truth OS. Configured with one TCP service listening (open port), one port deliberately closed, and one port dropped by a local firewall rule (filtered), so the scanner is exercised against all three states.
L2 Switch / Bridge	Ordinary switched LAN segment (VirtualBox host-only network or a physical switch) — keeps the attacker and victim on the same broadcast domain, matching the ARP/MAC-layer assumptions used elsewhere in the course project.
Defense / IDS Host	Optional third VM used in Section 7: sniffs mirrored traffic, flags the scan pattern, and applies TTL/window normalization at the gateway.

3 Timing Diagrams

Two phases are timed separately: the stealth port sweep, and the short, fixed-order OS-fingerprint probe battery that follows once at least one open and one closed port are known.

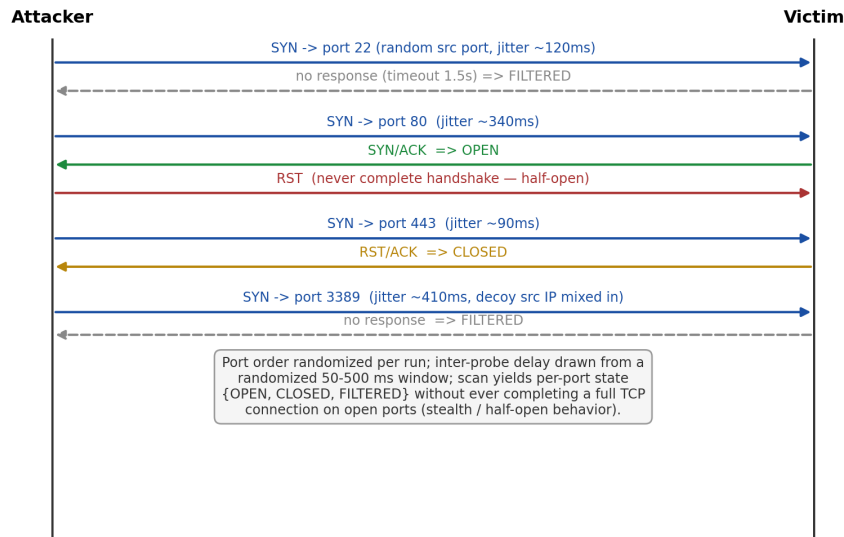
Figure 2 — Timing Diagram: Stealth (Half-Open) SYN Port Scan Phase

Figure 2: Half-open SYN scan: every open port receives an immediate RST instead of completing the handshake, so no application-level session is ever established on the victim.

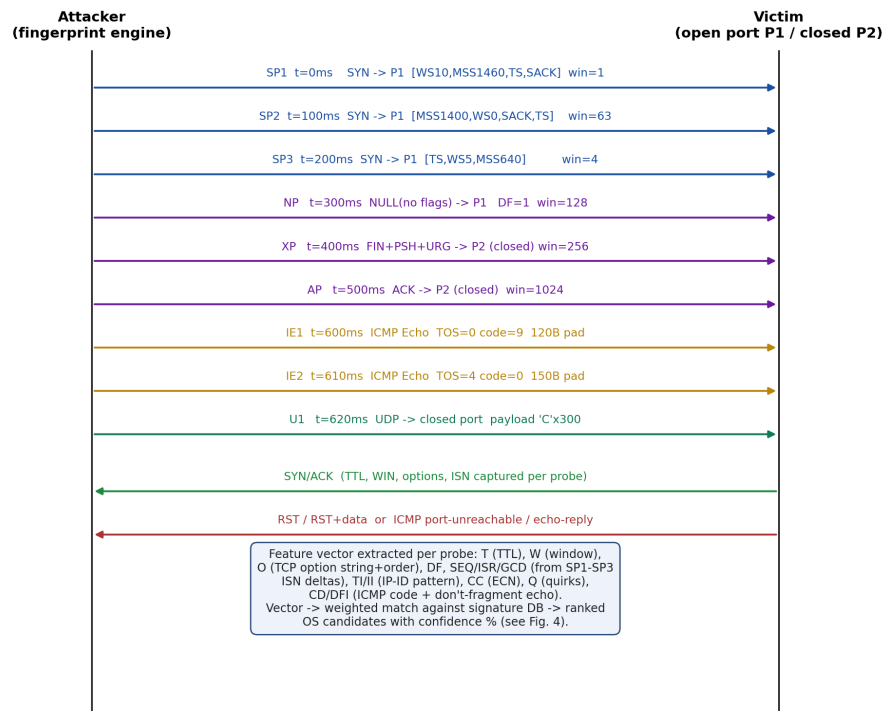
Figure 3 — Timing Diagram: Active OS-Fingerprint Probe Battery

Figure 3: Nine-probe OS-fingerprint battery. SP1–SP3 vary TCP options/order/window to expose stack-specific quirks and ISN growth rate; NP/XP/AP send RFC-edge-case flag combinations; IE1–IE2 and U1 add ICMP/UDP-derived TTL and ID-sequence signals.

4 Packet and Frame Structure

All packets are built field-by-field in Python (`struct.pack`) over raw sockets — no `scapy`/`nmap` calls. The fields below are the ones the tool actually reads or manipulates.

4.1 IP header fields used

Field	Size	Use in this tool
TTL	1 B	Read from every response; combined with hop-count estimate (from the U1 ICMP port-unreachable echo of the original TTL) to recover the victim's initial TTL — a primary OS discriminator (64 / 128 / 255 typical of Linux / Windows / network gear).
Identification (IP ID)	2 B	Sequence of IDs across probes is classified as zero / constant / incremental-by-1 / incremental-by-256 (“broken”, i.e. host byte order) / random — a strong OS/kernel discriminator.
DF (Don't Fragment)	1 bit	Recorded per probe and, for the two ICMP echo probes, compared pairwise (both set / both clear / echoed / toggled).
Protocol	1 B	Set to 6 (TCP), 1 (ICMP), or 17 (UDP) depending on probe.
Checksum	2 B	Computed manually (one's-complement sum) for every packet sent, since <code>IP_HDRINCL</code> disables kernel checksumming of a hand-built header.

4.2 TCP header fields and flags used

Field	Size	Use in this tool
Sequence / Ack number	4 B each	Random ISN chosen per SYN probe and recorded so the returned ISN can be measured; also used to classify the S/A response tests (echoed, echoed+1, zero, other).
Flags	9 bits	Varied per probe: SYN-only, no flags (NULL), FIN+PSH+URG (“Xmas-lite”), ACK-only, and SYN with ECE+CWR set for the ECN-style probe. Unusual combinations are what expose RFC-793 edge-case handling differences between stacks.
Window size	2 B	Recorded per probe; both the raw value and its variation across probes with different MSS values are discriminating.

Field	Size	Use in this tool
Options + order	variable	MSS, Window Scale, SACK-permitted, Timestamp and NOP/EOL are combined in a different order and value on each SYN probe (SP1–SP3); the exact order returned is rarely required by the RFC, so it varies by TCP/IP stack implementation and is one of the strongest signals used.
Urgent pointer / reserved bits	2 B / 3 bits	Checked for non-zero values that shouldn't be set (a “quirk” some stacks exhibit), recorded as the Q test.

4.3 TCP option encoding (as packed by the tool)

Option	Kind	Length	Value packed
EOL	0	1 B	Marks end of options list.
NOP	1	1 B	Padding / alignment, also used to perturb option order.
MSS	2	4 B	16-bit max-segment-size; SP1=1460, SP2=1400, SP3=640, matching values chosen to be distinguishable in captures.
Window Scale	3	3 B	8-bit shift count; varied 0 / 5 / 10 across the three SYN probes.
SACK-Permitted	4	2 B	No value field, presence/absence and position is the signal.
Timestamp	8	10 B	TSval fixed at 0xFFFFFFFF, TSecr = 0 on send; on receive, whether the TSval increases and at what rate (≈ 2 Hz / 100 Hz / 1000 Hz) is measured across the battery.

4.4 Probe battery sent (own reduced set, informed by Nmap's approach)

#	Probe	Target	Crafted characteristics
SP1	TCP SYN	open port	WS(10), NOP, MSS(1460), TS(0xFFFFFFFF,0), SACK-permitted; window field = 1.
SP2	TCP SYN	open port	MSS(1400), WS(0), SACK, TS, EOL; window field = 63.
SP3	TCP SYN	open port	TS, NOP, NOP, WS(5), NOP, MSS(640); window field = 4.
NP	TCP, no flags set	open port	DF set, window field = 128 — RFC 793 does not define this case explicitly.

#	Probe	Target	Crafted characteristics
XP	TCP	closed port	DF clear, window field = 256 — illegal flag combination, response varies by stack.
AP	TCP	closed port	DF set, window field = 1024 — expect RST; window/TTL of the RST recorded.
IE1 / IE2	ICMP	host	TOS 0 / TOS 4, code 9 / code 0, 120 B / 150 B payload, sent 10 ms apart; DF and response code checked for RFC-violating echoes.
U1	UDP	closed port	300 bytes of 0x43 ('C'); expects ICMP port-unreachable, whose embedded original-TTL is used to compute hop distance and back out the victim's initial TTL.

5 Implementation Plan

5.1 Tooling and constraints

- Language: Python 3, using `socket.AF_INET / SOCK_RAW` with `IP_HDRINCL` for full manual IP+TCP/UDP/ICMP header construction (`struct.pack`), plus a raw `AF_PACKET` sniffing thread to capture and correlate responses. No scapy, no nmap, no external scanning library — everything from checksum to option bytes is packed by hand, as required.
- Run as root/Administrator (required for raw sockets on both Linux and Windows/Npcap).
- Target lab OSES for calibration: a Linux distribution, Windows 10/11, and (if available) a BSD or embedded/router image, to seed a signature database with real captured fingerprints rather than only textbook values.

5.2 Implementation approach

The tool will be organized around four conceptual responsibilities rather than a fixed set of files. *Packet construction* covers building raw IP, TCP, UDP, and ICMP headers byte-by-byte, computing one's-complement checksums by hand, and encoding TCP options in a specified order — this is the shared foundation both the scanner and the fingerprint engine build on. *Stealth scanning* covers randomizing the port list and inter-probe delay, sending SYN probes, and classifying each port as OPEN, CLOSED, or FILTERED from the response pattern, optionally interleaving decoy source addresses. *OS fingerprinting* covers sending the fixed nine-probe battery described in Section 4.4 to a confirmed open and a confirmed closed port, at the precise inter-probe timing needed for the ISN/timestamp-rate measurements to be valid, and turning the captured responses into the attribute vector (T, W, O, DF, GCD/ISR/SP, TI/II/CI, CC, Q, CD, DFI) described in Section 4. *Matching and reporting* covers scoring that attribute vector against a self-built signature database — adding each attribute's weight to a candidate OS's score when the observed value falls inside its expected range or matches its expected category — normalizing to a confidence percentage, sorting candidates, and printing the ranked table, probability bar, and summary block shown in Figure 5.

A background thread will continuously sniff inbound traffic on the raw socket for the duration of

each scan or probe battery, matching responses back to the specific probe that produced them (by source/destination port, sequence/acknowledgment number, or ICMP identifier/sequence as appropriate) so that out-of-order or delayed responses do not get attributed to the wrong probe.

5.3 Matching pipeline

Figure 4 — OS-Fingerprint Matching Pipeline

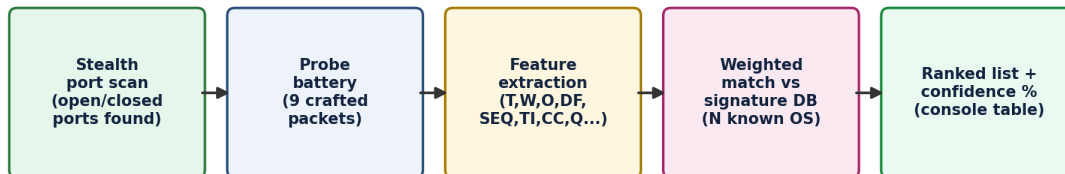


Figure 4: End-to-end flow from stealth scan through to the ranked, confidence-scored OS list.

5.4 Stealth scan algorithm (summary)

- Shuffle the target port list; for each port, wait a random jitter in [50, 500] ms, then send one SYN with a random ephemeral source port.
- If SYN-ACK is received: mark OPEN, immediately send RST (never call `connect()`; no full handshake, nothing reaches the victim’s application layer or its connection log).
- If RST/ACK is received: mark CLOSED.
- If nothing is received after a timeout (with one retry): mark FILTERED.
- Optional decoy mode: interleave a few SYNs from spoofed source IPs alongside the real scanning IP so a naive per-source-IP counter under-counts the real attacker’s share of the traffic.

5.5 OS-detection algorithm (summary)

- Once one OPEN and one CLOSED port are known, send the 9-probe battery in fixed order and fixed spacing (Figure 3).
- Extract the attribute vector described in Section 4 / Figure 3.
- Score every OS entry in the signature database: for each attribute the entry defines, add its weight to the score if the observed value matches (exact match for categorical fields such as O/Q/CC, range match for numeric fields such as T and W).
- Confidence % = score / max attainable score for that entry $\times 100$; sort descending; print the top-N rows as a ranked table with an ASCII probability bar, matching Figure 5.

5.6 Testing and calibration plan

- Stand up 3–4 known-OS VMs on the lab LAN; run the probe battery against each and record the resulting vectors as signature-database entries — this is how the database in Figure 4 is

actually built, not copied from a public list.

- Benchmark against Nmap: run `nmap -O` against the same calibration VMs and record its top candidate and confidence. Compare Nmap’s guess, this tool’s guess, and the VM’s actual installed OS side by side for each target, and report agreement rate (this tool vs. actual) and (this tool vs. Nmap) as the accuracy metrics in the Final Report.
- Verify the stealth scanner never appears in the victim’s completed-connection log (e.g. no entry in a simple TCP echo server’s `accept()` log) for ports it reports OPEN.

6 Expected Outcome of a Successful Attack

A successful run discovers the victim’s true port state (open/closed/filtered) for every scanned port without completing a TCP handshake on any open port, and produces a ranked OS-candidate table with confidence percentages, an ASCII probability bar for the top result, and a short summary block — the format the tool is being built to reproduce is shown below, which has been provided by the supervisor.

Rank	Operating System	Confidence	Probability Bar
1	Sun Solaris 9	69.13%	
2	Android 5.0.1	67.46%	
3	DD-WRT v24-sp2 (Linux 2.4.36)	67.46%	
4	TP-LINK TL-WA801ND vAP (Linux 2.6.36)	67.46%	
5	Microsoft Windows Server 2012 R2	64.41%	
6	Linux 2.6.36	64.07%	
7	Linux 2.6.32	63.05%	
8	Minix 2.0.4	63.05%	
9	Oracle Solaris 11	62.37%	
10	Sun Solaris 10	62.37%	
11	Sun Solaris 10 (SPARC)	62.37%	
12	Sun Solaris 8 (SPARC)	62.37%	
13	Sun Solaris 9 or Oracle Solaris 10	62.37%	
14	Linux 2.6.31	61.02%	
15	Microsoft Windows Embedded Standard 7	61.02%	
16	Microsoft Windows Phone 7.5 or 8.0	61.02%	
17	FreeBSD 11.0-RELEASE	60.68%	
18	Android 2.3.7 (Linux 2.6.37)	60.68%	
19	DD-WRT v24-sp2 (Linux 2.4.37)	60.68%	
20	DD-WRT v23 (Linux 2.4.34)	60.68%	
21	DD-WRT v24-sp1 (Linux 2.4)	60.68%	
22	OpenWrt Kamikaze 8.09 (Linux 2.6.25.20)	60.68%	
23	IPCop 1.9.19 or IPFire 2.9 firewall (Linux 2.6.32)	60.68%	
24	IPFire 2.11 firewall (Linux 2.6.32)	60.68%	
25	IPCop 2.0 (Linux 2.6.32)	60.68%	

Source IP: 172.24.137.33

Summary

Best Match: Sun Solaris 9
 Confidence: 69.13%
 Showing: 15 of 40 matches
 Use --debug to see all results

Figure 5: Target CLI output format: ranked OS candidates, confidence percentage, probability bar, source IP, and a top-match summary (`--debug` reveals the full candidate list).

7 Defense Idea

7.1 Detecting the scan

- Signature-based: flag any inbound TCP segment with an RFC-anomalous flag combination (NULL, all-flags/“Xmas”, SYN+FIN together) from the same source within a short window — these almost never occur in benign traffic and directly match probes NP/XP in the battery.
- Rate/behavior-based: count distinct destination ports probed per source IP in a sliding time window (e.g. netfilter’s hashlimit/recent modules, or a small custom sniffer); many-ports/short-window/no-data-exchanged is the stealth-scan signature.
- Log SYNs that are answered but never followed by a completed handshake from the same source — a hallmark of half-open scanning.

7.2 Weakening OS-fingerprint accuracy specifically

- TTL normalization at the gateway/host (e.g. `iptables -t mangle -j TTL --ttl-set 64`) removes the T/TG signal regardless of the real OS.
- Reorder or strip TCP options on egress (a small netfilter queue handler rewriting the options field) to defeat the O/O1–O6 test, which is one of the strongest discriminators used.
- Disable or rate-limit ICMP timestamp/address-mask/echo replies, and keep SYN cookies + kernel ISN randomization enabled (default on modern kernels) to weaken SEQ/ISR/SP and II/CD tests.
- Deploy a decoy responder (portspooft-style) on unused ports that answers scans with deliberately inconsistent banners/TCP behavior, poisoning the attacker’s confidence scores with false candidates.

7.3 Mitigating / blocking the scan itself

- Connection-rate limiting (`iptables --connlimit / hashlimit`) to slow a scan below the speed needed to complete the port sweep before triggering a block.
- Auto-block: once a source IP crosses the distinct-port threshold in the detection window, insert a temporary DROP rule for that source — proposed as the bonus defense-mechanism component, implemented as a small always-on daemon sniffing SYNs and driving iptables.

References

1. G. Lyon (“Fyodor”), “TCP/IP Fingerprinting Methods Supported by Nmap,” *Nmap Network Scanning*, <https://nmap.org/book/osdetect-methods.html>.
2. E. Hjelmvik, “Passive OS Fingerprinting,” *Netresec Blog*, Nov. 2011, <https://www.netresec.com/?page=Blog&month=2011-11&post=Passive-OS-Fingerprinting>.
3. RFC 793 — Transmission Control Protocol; RFC 1122 — Requirements for Internet Hosts; RFC 3168 — The Addition of Explicit Congestion Notification (ECN) to IP.